

# Hybrid Search in PostgreSQL

Combining Vector and Full-Text for Real-World Applications

Shayon Sanyal, Principal PostgreSQL Specialist SA

Postgres Summit US 2026, New York City  
September 30, 10:30–11:20 EDT, Letterpress

# Two questions. Two different misses.

## Keyword search misses

**"Where should I park my rainy-day / emergency fund?"**

The best answer is **#2** for vector search. Keyword search ranks it **#29**, BM25 **#40**, and requiring every word finds nothing.

## Vector search misses

**"Employer rollover from 403b to 401k?"**

The only answer is **#1** for BM25 and **#26** for vector search. Exact identifiers are what keyword search is for.

**Words find exact terms. Vectors find paraphrases.** Which one wins, and whether combining them helps, is a measurement, not a belief.



About me

# Shayon Sanyal

**Principal PostgreSQL Specialist Solutions Architect**

I help teams build on PostgreSQL, from relational applications to retrieval and agent workflows.

Today: readable SQL, measured results, and a skill you can point at your own tables.

[linkedin.com/in/shayonsanyal](https://linkedin.com/in/shayonsanyal)

How we'll measure

## 2,271 questions with known right answers.

**FiQA-2018:** 57,638 finance forum posts, 648 test questions. The main example.

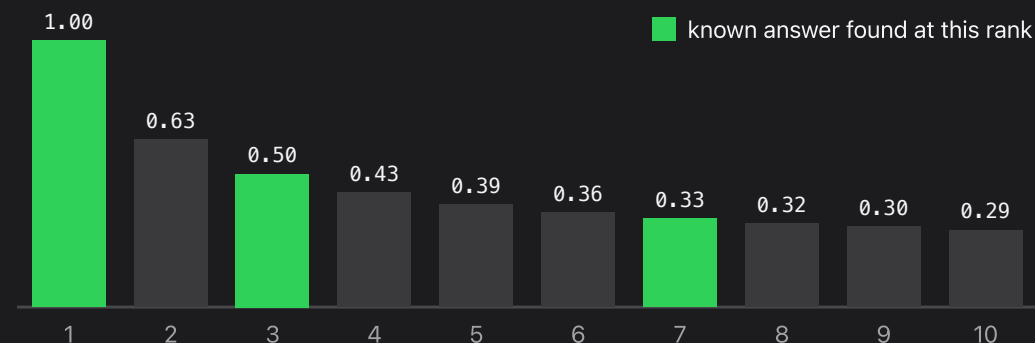
**SciFact, NFCorpus, SCIDOCS:** chosen by one rule before measuring: public BEIR benchmark sets under 30,000 documents where keyword search (BM25) beat every vector method in 2021.

**NDCG@10** scores a ranking higher when the known answers sit near the top.

The abstract promised that combining beats either alone. **We'll measure when it does, and when it doesn't.**

Credit for a known answer at each rank

$1 / \log_2(\text{rank} + 1)$ : an answer at rank 1 is worth more than three answers at rank 10



## Four datasets, four kinds of question

| DATASET: WHAT IT SEARCHES                | A TYPICAL TEST QUESTION                                                                    | TEST QUESTIONS | KNOWN ANSWERS PER QUESTION | BM25 VS VECTOR (EMBED V4), NDCG@10 |
|------------------------------------------|--------------------------------------------------------------------------------------------|----------------|----------------------------|------------------------------------|
| <b>FiQA:</b> 57,638 finance forum posts  | "Where should I park my rainy-day / emergency fund?"                                       | 648            | 2.6                        | 23.6 vs 53.9                       |
| <b>SciFact:</b> 5,183 science abstracts  | "Anthrax spores can be disposed of easily after they are dispersed." A claim to check      | 300            | 1.1                        | 68.8 vs 77.5                       |
| <b>NFCorpus:</b> 3,633 medical abstracts | "low-carb diets." Half are two words or fewer                                              | 323            | 38.2, graded               | 32.3 vs 40.1                       |
| <b>SCIDOCS:</b> 25,657 paper abstracts   | "A Fast Learning Algorithm for Deep Belief Nets." A title; its answers are papers it cites | 1,000          | 4.9                        | 15.4 vs 20.6                       |

Same methods, very different gaps: BM25 reaches **89%** of vector search on SciFact but **44%** on FiQA. One dataset would have told a different story.

# The stack: one PostgreSQL database

| PIECE                             | VERSION   | JOB                                                                                              |
|-----------------------------------|-----------|--------------------------------------------------------------------------------------------------|
| PostgreSQL                        | 18.6      | <code>tsvector</code> , GIN, <code>ts_rank_cd</code> , the SQL that combines and scores results  |
| pgvector                          | 0.8.6     | <code>vector(1536)</code> , HNSW, iterative index scans, <code>halfvec</code> , <code>bit</code> |
| pg_textsearch                     | 1.4.0     | BM25 index and <code>&lt;@&gt;</code> operator (PostgreSQL license)                              |
| Cohere Embed v4 on Amazon Bedrock | 1536 dims | Posts as <code>search_document</code> , questions as <code>search_query</code>                   |
| Cohere Rerank on Amazon Bedrock   | 3.5       | Reorders a top 50 outside the database                                                           |
| bge-small-en-v1.5 with fastembed  | 384 dims  | A small open-source model, on this laptop                                                        |
| VS Code + SQLTools                |           | Numbered SQL files you can run yourself                                                          |

The database side is all open source: PostgreSQL, pgvector, pg\_textsearch. So is bge-small, and that fully open path is where hybrid pays most. Every number today is from this laptop.

## One row, three representations 01\_schema.sql

```
CREATE TABLE docs (  
  id          text PRIMARY KEY,  
  body        text NOT NULL,  
  tsv         tsvector GENERATED ALWAYS AS (to_tsvector('english', body)) STORED,  
  embedding   vector(1536), -- Cohere Embed v4, input_type = 'search_document'  
  embedding_local vector(384) -- bge-small-en-v1.5, open source, local  
);  
  
CREATE INDEX ON docs USING gin (tsv); -- 0.6 s  
CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops); -- 30.4 s  
CREATE INDEX ON docs USING bm25 (body) WITH (text_config = 'english'); -- 3.9 s
```

The lexemes are **generated** and cannot drift from the text. Embeddings are not: **one column per model**, re-embedded when the text changes.

## Pitfall 1: every word must match 03

```
SELECT websearch_to_tsquery('english', 'Where should I park my rainy-day / emergency fund?');  
-- 'park' & 'rainy-day' <=> 'raini' <=> 'day' & 'emerg' & 'fund'
```

`websearch_to_tsquery` and `plainto_tsquery` join words with **AND**. A natural question rarely has all its words in one answer.

**405 of 648 questions match no post at all. NDCG@10: 4.3**

## Match any word, rank the matches 04

```
WITH question AS (  
    SELECT replace(plainto_tsquery('english', body)::text, ' & ', ' | ')::tsquery AS tsq  
    FROM active_query  
)  
SELECT d.id, ts_rank_cd(d.tsv, q.tsq) AS score  
    FROM docs d, question q  
WHERE d.tsv @@ q.tsq  
ORDER BY score DESC  
LIMIT 50;
```

Recall@50 (known answers anywhere in the top 50) doubles, 6.7 → 14.6, but NDCG@10 **falls to 2.8**: with no IDF, “fund” and “day” outweigh “rainy-day”. The rainy-day question matches **10,460 posts**, and `ts_rank_cd` scores every one of them: **81 ms**.

## ts\_rank\_cd is not BM25 05

```
SELECT id, -(body <@> to_bm25query(:'question', 'docs_body_bm25')) AS bm25
FROM docs
ORDER BY body <@> to_bm25query(:'question', 'docs_body_bm25')
LIMIT 50;
```

BM25 weighs rare terms up (IDF), saturates repeats, and normalizes for length. The index keeps the corpus statistics and returns the top 50 **without scoring every match**. 23.6 is exactly BEIR's published BM25 score for FiQA (0.236).

| KEYWORD METHOD       | NDCG@10 | TOP 50, RAINY-DAY QUESTION |
|----------------------|---------|----------------------------|
| ts_rank_cd, any word | 2.8     | 81 ms                      |
| BM25, pg_textsearch  | 23.6    | 0.27 ms                    |

## Vector search: the top 50 by meaning 06

```
SET hnsw.ef_search = 100;           -- default 40: LIMIT 50 would silently return 40
SELECT id, 1 - (embedding <=> (SELECT embedding FROM active_query)) AS cosine
FROM docs
WHERE embedding IS NOT NULL
ORDER BY embedding <=> (SELECT embedding FROM active_query)
LIMIT 50;
```

Keep the distance operator in `ORDER BY`, ascending, with a `LIMIT`. Embed questions as `search_query` and posts as `search_document`.

NDCG@10 **53.9**, median **3.5 ms** per question

## Pitfall 2: adding scores from different scales 07a 07b

| RAINY-DAY QUESTION, EACH LIST'S TOP 50              |  | MIN   | MAX         |
|-----------------------------------------------------|--|-------|-------------|
| ts_rank_cd                                          |  | 1.70  | 5.40        |
| cosine similarity                                   |  | 0.478 | 0.598       |
| HOW THE TWO LISTS ARE COMBINED                      |  |       | NDCG@10     |
| keyword_score + cosine                              |  |       | 5.6         |
| keyword list, then vector list, duplicates removed  |  |       | 2.9         |
| <b>Reciprocal Rank Fusion</b> of the same two lists |  |       | <b>33.8</b> |

Same inputs, 6× the score. `ts_rank_cd`'s normalization options keep its scores in a range, but still not comparable to cosine.

# Combine ranks, not scores

$$\text{RRF}(d) = \sum 1 / (60 + \text{rank}_{\text{list}}(d))$$

| BM25                        | VECTOR                      | COMBINED BY RRF             |
|-----------------------------|-----------------------------|-----------------------------|
| 1 66626                     | 1 293687                    | 1 <b>32811</b> known answer |
| 2 371922                    | 2 348514                    | 2 371922 0.02912            |
| 3 569342                    | 3 469809                    | 3 469809 0.02778            |
| 4 <b>32811</b> known answer | 4 361639                    | 4 424427 0.02633            |
| 5 2128                      | 5 458063                    | 5 32671 0.02568             |
| 6 32671                     | 6 427365                    | 6 72960 0.02452             |
| 7 404800                    | 7 209789                    | 7 488737 0.02379            |
| 8 266457                    | 8 283692                    | 8 152096 0.02114            |
| 9 272840                    | 9 <b>32811</b> known answer | 9 293687 0.01639            |
| 10 65567                    | 10 62897                    | 10 66626 0.01639            |

Known answer: BM25 #4, vector #9 →  $1/64 + 1/69 = \mathbf{0.03012}$ , combined #1.

Each list's own #1 is missing from the other list:  $1/61 = 0.01639$ , combined #9 and #10.

## RRF is a full outer join 08a

```
WITH keyword AS (  
    SELECT id, row_number() OVER (ORDER BY ts_rank_cd(tsv, q) DESC, id) AS rank  
    FROM docs, (SELECT replace(plainto_tsquery('english', :'question')::text,  
                                ' & ', ' | ')::tsquery AS q) question  
    WHERE tsv @@ q ORDER BY rank LIMIT 50  
,  
semantic AS (  
    SELECT id, row_number() OVER (ORDER BY distance, id) AS rank  
    FROM (SELECT id, embedding <=> :'question_embedding' AS distance  
          FROM docs WHERE embedding IS NOT NULL  
          ORDER BY distance LIMIT 50) nearest  
)  
SELECT coalesce(k.id, v.id) AS id,  
       coalesce(1.0 / (60 + k.rank), 0) + coalesce(1.0 / (60 + v.rank), 0) AS rrf  
FROM keyword k FULL OUTER JOIN semantic v USING (id)  
ORDER BY rrf DESC  
LIMIT 10;
```

One statement, two indexes (GIN and HNSW). Median **150 ms** with `ts_rank_cd`; **8.2 ms** with BM25 (08b). The two lists run **one after the other** in one backend process, not in parallel: the plan has no Gather node, and 8.2 ms is about BM25's 5.2 plus vector's 3.5.

## Blend scores the right way 08c

```
keyword_norm AS (
    -- per question: min-max to [0, 1]; semantic_norm is the same
    SELECT id, (score - min(score) OVER ()) / (max(score) OVER () - min(score) OVER ()) AS s
    FROM keyword
)
SELECT id, w * coalesce(v.s, 0) + (1 - w) * coalesce(k.s, 0) AS blend
FROM keyword_norm k FULL OUTER JOIN semantic_norm v USING (id);
```

RRF ignores how **strong** each match is. A blend keeps it: scale each list's scores to 0–1, then weight them. It is what “adding scores” should have been in pitfall 2.

| WEIGHT <b>w</b> ON VECTOR, CHOSEN ON SEPARATE TUNING QUESTIONS | FIQA | SCIFACT | NFCORPUS | SCIDOCs        |
|----------------------------------------------------------------|------|---------|----------|----------------|
| Cohere Embed v4 (frontier)                                     | 1.0  | 0.70    | 0.70     | 0.5, not tuned |
| bge-small (local)                                              | 0.65 | 0.55    | 0.65     | 0.5, not tuned |

# Knobs that change different things

| KNOB                              | CONTROLS                             | WATCH                                                                                                            |
|-----------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Results per list</b> (50)      | How many posts each list contributes | A post outside both lists can't be combined or reranked:<br><b>Recall@50</b>                                     |
| <b>k</b> (60)                     | How fast credit falls with rank      | Measured k = 5 to 200: at most 4.3 NDCG@10 (FiQA), 1.4 elsewhere. No k made equal-weight RRF beat Embed v4 alone |
| <b>Weights</b>                    | Trust in each list                   | Equal-weight RRF lost to vector on all four datasets; tune a blend's weight on separate questions instead        |
| <code>hnsw.ef_search</code> (100) | How hard each HNSW scan searches     | Must be $\geq$ results per list                                                                                  |

Change one at a time, and re-run the test questions.

Live on NFCorpus: all local except the last column

# Small model + BM25 beat the frontier model here

NFCorpus test question PLAIN-307, with 3 known answers, also active in VS Code

## Vitamin D: Shedding some light on the new recommendations

**A** in 4 of 4 top 10s   **B** in 3 of 4 top 10s   **C** in 4 of 4 top 10s

BM25

SQL

65

NDCG@10

3 of 3 answers in the top 10  
10.5 ms, live SQL

- 1 Safety considerations and potential interactions of vitamins: should vitamins be considered drugs?...
- 2 **B** VITAMIN D: A D-LIGHTFUL SOLUTION FOR HEALTH Throughout evolution sunlight produced vitamin D in the ski...
- 3 The aryl hydrocarbon receptor and its xenobiotic ligands: a fundamental

Vector (bge-small, local)

SQL

77

NDCG@10

2 of 3 answers in the top 10  
12.8 ms, live SQL

- 1 **A** Towards prevention of vitamin D deficiency and beyond: knowledge gaps and research needs in vitamin D...
- 2 **C** Evaluation, treatment, and prevention of vitamin D deficiency: an Endocrine Society clinical practice guideline....
- 3 Traditionally living populations in East

Tuned blend (bge-small)

SQL

97

NDCG@10

Highest

3 of 3 answers in the top 10  
15.7 ms, live SQL

- 1 **C** keyword 4 vector 2 Evaluation, treatment, and prevention of vitamin D deficiency: an Endocrine Society clinical practice guideline....
- 2 **A** keyword 8 vector 1 Towards prevention of vitamin D deficiency and beyond: knowledge gaps and research needs in vitamin D...
- 3 keyword 14 vector 4

Vector (Embed v4)

SQL

71

NDCG@10

3 of 3 answers in the top 10  
21.5 ms, live SQL

- 1 Low Vitamin D Status: Definition, Prevalence, Consequences and Correction Vitamin D is obtained fro...
- 2 **A** Towards prevention of vitamin D deficiency and beyond: knowledge gaps and research needs in vitamin D...
- 3 **B** VITAMIN D: A D-LIGHTFUL SOLUTION

localhost:8018, pick NFCorpus, then “Local hybrid beats the frontier model”

# From text to vector

- **Cohere Embed v4** on Amazon Bedrock: 1536 dimensions, unit-length vectors, so cosine distance and inner product rank the same way.
- **Two input types.** Posts are embedded as `search_document`, questions as `search_query`. Swapping them lowers relevance without any error.
- **One model per column.** Vectors from different models or versions are not comparable, even at the same dimension. Store the model id.
- **A small local model too:** bge-small-en-v1.5, 384 dims, open source, in its own column. FiQA's 57,638 posts took about an hour on the laptop CPU; Embed v4 took 35 minutes behind a tokens-per-minute quota. Either way, a rerun fills only the rows still NULL.

## Pitfall 3: a filter is not a pre-filter 09

```
SELECT id FROM docs
WHERE tsv @@ plainto_tsquery('english', 'mortgage') -- 4.2% of posts
ORDER BY embedding <=> : 'question_embedding' LIMIT 10;
```

| SETTING                                                                                      | PLAN                                                     | ROWS<br>RETURNED |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------|------------------|
| defaults ( <code>iterative_scan = off</code> , <code>ef_search</code><br><code>= 40</code> ) | HNSW, then filter: <b>Rows Removed by<br/>Filter: 39</b> | <b>1 of 10</b>   |
| <code>SET hnsw.iterative_scan = relaxed_order</code>                                         | HNSW keeps walking until 10 pass                         | 10 of 10         |
| a rare term: <code>he loc</code> (0.2%)                                                      | GIN, then exact sort                                     | 10 of 10         |

pgvector 0.8 iterative scans are bounded by `hnsw.max_scan_tuples` (20,000).

## Similar to the question AND contains a word 11

```
SELECT h.*, left(d.body, 60)
FROM hybrid_search(:'question', :'question_embedding',
                  match_count => 5, required_terms => 'mortgage') h
JOIN docs d ON d.id = h.doc_id;
```

```
CREATE FUNCTION hybrid_search(...) ... LANGUAGE sql STABLE
SET hnsw.ef_search = 100
SET hnsw.iterative_scan = relaxed_order
SET plan_cache_mode = force_custom_plan -- keeps "required_terms IS NULL OR ..." indexable
```

The required word goes into **both** lists, before their LIMIT.

# Boost and expand: two patterns on top of `hybrid_search()`

**Boost** by recency, popularity, preference

```
SELECT h.doc_id,
       h.score
  * power(0.5, extract(epoch FROM
                      now() - d.published_at) / 86400 / 30)
  * (1 + ln(1 + d.likes) / 10)
  * CASE WHEN d.category = ANY(:'preferred')
        THEN 1.2 ELSE 1 END AS boosted
FROM hybrid_search(:'question',
                  :'question_embedding', match_count => 50) h
JOIN docs d ON d.id = h.doc_id
ORDER BY boosted DESC
LIMIT 10;
```

**Expand** along links, two hops

```
WITH RECURSIVE related (doc_id, score, depth) AS (
  SELECT doc_id, score, 0
    FROM hybrid_search(:'question',
                      :'question_embedding', match_count => 10)
  UNION ALL
  SELECT l.dst, r.score / 2, r.depth + 1
    FROM related r
   JOIN links l ON l.src = r.doc_id
  WHERE r.depth < 2
)
SELECT doc_id, max(score) AS score
  FROM related
 GROUP BY doc_id
 ORDER BY score DESC
LIMIT 10;
```

Boost a larger pool than you show (50, then keep 10), or a boost can't lift anything. **Not measured here:** these datasets have no dates, popularity, or links.

## Hybrid vs vector, same embedding model: NDCG@10 change on 2,271 test questions

|                                                           | FIQA                            | SCIFACT                      | NFCORPUS                     | SCIDOCS                        |
|-----------------------------------------------------------|---------------------------------|------------------------------|------------------------------|--------------------------------|
| Small local model: bge-small, 384 dims, on this laptop    |                                 |                              |                              |                                |
| Tuned blend + BM25                                        | <b>+1.2 ↑</b><br>+0.4...+2.1    | <b>+2.1 ↑</b><br>+0.0...+4.3 | <b>+2.1 ↑</b><br>+1.3...+3.0 | +0.2 *<br>-0.5...+0.8          |
| Equal-weight RRF + BM25                                   | <b>-3.1 ↓</b><br>-4.7...-1.5    | +1.7<br>-0.9...+4.3          | <b>+2.3 ↑</b><br>+1.0...+3.7 | -0.3<br>-1.0...+0.5            |
| Frontier model cut to its first 256 dims: Cohere Embed v4 |                                 |                              |                              |                                |
| Tuned blend + BM25                                        | +0.5<br>-0.0...+1.1             | <b>+3.5 ↑</b><br>+1.3...+5.7 | <b>+2.8 ↑</b><br>+1.5...+4.1 | <b>+1.9 ↑ *</b><br>+1.3...+2.5 |
| Frontier model: Cohere Embed v4, 1536 dims                |                                 |                              |                              |                                |
| Tuned blend + BM25                                        | 0.0<br>chose vector only        | +0.2<br>-1.2...+1.4          | <b>+0.8 ↑</b><br>+0.1...+1.5 | <b>-0.8 ↓ *</b><br>-1.4...-0.2 |
| Equal-weight RRF + BM25                                   | <b>-12.5 ↓</b><br>-14.5...-10.6 | <b>-2.9 ↓</b><br>-5.1...-0.6 | -1.1<br>-2.2...+0.1          | <b>-1.2 ↓</b><br>-2.0...-0.5   |
| Cohere Rerank 3.5, top 50                                 | <b>-4.1 ↓</b><br>-6.0...-2.3    | -0.4<br>-2.8...+1.9          | <b>-1.9 ↓</b><br>-3.3...-0.4 | -0.4<br>-1.1...+0.3            |

↑ / ↓: 95% confidence range entirely above / below zero, so not noise. \* SCIDOCS: no tuning questions, weight 0.5

## When hybrid pays: a smaller model

| NDCG@10                                                                  | FIQA        | SCIFACT     | NFCORPUS    | SCIDOCs      |
|--------------------------------------------------------------------------|-------------|-------------|-------------|--------------|
| bge-small alone (local, open source)                                     | 38.0        | 72.0        | 33.8        | 19.6         |
| <b>bge-small + BM25, tuned blend (all local)</b>                         | <b>39.2</b> | <b>74.2</b> | <b>35.9</b> | 19.8         |
| bge-small + <code>ts_rank_cd</code> , tuned blend (core PostgreSQL only) | 37.9        | 72.4        | 34.4        | 16.0 ↓       |
| Cohere Embed v4 alone (frontier API)                                     | 53.9        | 77.5        | 40.1        | 20.6         |
| <b>Share of the gap closed by BM25</b>                                   | 8%          | <b>39%</b>  | <b>33%</b>  | within noise |

PostgreSQL, BM25, and a 384-dimension model on one laptop, no API calls: adding keyword search closes **a third or more of the gap** to a frontier model on SciFact and NFCorpus. Without BM25's IDF, `ts_rank_cd` gained nothing beyond noise.

# Rerank on FiQA: measure it too

| WHAT COHERE RERANK 3.5 REORDERS | NDCG@10 | RECALL@50 | MEDIAN   |
|---------------------------------|---------|-----------|----------|
| none: vector alone              | 53.9    | 78.5      | 3.5 ms   |
| vector top 50                   | 49.7    | 78.5      | 417 ms   |
| RRF (BM25) top 50               | 50.4    | 75.2      | 1,093 ms |
| vector $\cup$ BM25, up to 100   | 49.0    | 76.5      | 1,208 ms |

It lifts 403b from #26 to #2, but puts a known answer first on 47.5% of questions against 53.9% for Embed v4 alone. On no dataset did a reranked method beat vector search by more than noise.

- A reranker reads the question and each result **together**, but can only reorder what the lists found: Recall@50 is its ceiling.
- It runs **outside PostgreSQL**: a network call per question, and the results' text leaves the database.

# Storage on FiQA: fewer bits or fewer dimensions?

| INDEX ON EMBEDDING                                               | BYTES PER VECTOR | HNSW SIZE     | NDCG@10 | MEDIAN MS |
|------------------------------------------------------------------|------------------|---------------|---------|-----------|
| <code>vector(1536)</code>                                        | 6,148            | 450 MB        | 53.9    | 3.5       |
| <code>halfvec(1536)</code> expression index                      | 3,080            | 225 MB        | 53.7    | 3.4       |
| <code>binary_quantize()</code> , then re-sort 200 by full vector | 200              | 28 MB         | 54.1    | 3.3       |
| first 1024 dims, <code>subvector()</code>                        | 4,104            | <b>450 MB</b> | 53.4    | 2.9       |
| first 512 dims                                                   | 2,056            | 150 MB        | 51.8 ↓  | 2.2       |
| first 256 dims                                                   | 1,032            | 75 MB         | 49.3 ↓  | 1.7       |

**Quantize before you truncate.** Fewer bits kept NDCG within noise; 512 and 256 dims lost 1.6 to 5.5 points on every dataset. And 1024 dims saves no index space: an 8 KB page still holds one vector. If you do truncate, add BM25: at 256 dims it won back half the loss or more on three datasets.

# What these numbers do and don't prove

| THEY SHOW                                                       | THEY DON'T SHOW                                                            |
|-----------------------------------------------------------------|----------------------------------------------------------------------------|
| Which method ranks known answers higher on four public datasets | That the same holds for your data                                          |
| The pitfalls, reproduced with real plans                        | Production latency: this is one laptop, one client                         |
| Relative cost of each method on up to 57,638 documents          | Behavior at 50 million rows or under concurrency                           |
| This embedding and rerank model pair                            | Future models: they improve, and results depend on the model versions used |
| A public benchmark                                              | That the models never saw similar text in training                         |
| Two embedding models, one reranker                              | How other models behave; SCIDOCS blends aren't tuned (no tuning questions) |
| English questions, word matching                                | Multilingual queries or phrase search ( <-> )                              |
| Boost and link-expansion patterns                               | Their effect: these datasets have no dates, popularity, or links           |

The judgments are incomplete: an unjudged document can be a good answer and still count as a miss.

Three things to take home

**Rank words with BM25.  
Blend, tuned on separate questions.  
Hybrid pays most on smaller models.**

Local model + BM25: **+1.2 to +2.1 NDCG@10 on three of four datasets.** Frontier model: equal-weight RRF never won, not even on FiQA's 177 questions with a number or acronym (vector 57.7, RRF 46.5). **Measure on your own questions before you choose.**

# Take it home

**The lab:** `hybrid-lab/`. Numbered SQL, VS Code cells, the UI, and the scoreboard.

**The skill:** adds hybrid search to *your* table and measures it.

```
/plugin marketplace add shayons/talks  
/plugin install postgres-hybrid-search@shayons-talks
```

Other agents: copy `hybrid-search-  
plugin/skills/postgres-hybrid-search/`.

[github.com/shayons/talks](https://github.com/shayons/talks), in [conferences/2026-postgresconf-agentic-ai](#)



# Thank you.

Which of your queries needs both words and meaning?

Shayon Sanyal, [linkedin.com/in/shayonsanyal](https://www.linkedin.com/in/shayonsanyal)

[github.com/shayons/talks](https://github.com/shayons/talks)

# References

- **PostgreSQL 18:** [Full-text search, Using EXPLAIN, auto\\_explain](#)
- **pgvector 0.8.6:** [HNSW, filtering, iterative index scans](#); **pg\_textsearch 1.4.0:** [BM25 for PostgreSQL](#)
- **RRF:** Cormack, Clarke & Büttcher, [SIGIR 2009](#); **HNSW:** Malkov & Yashunin, [arXiv:1603.09320](#)
- **Score blending:** Bruch, Gai & Ingber, "An Analysis of Fusion Functions for Hybrid Retrieval", [ACM TOIS 2023](#); **bge-small-en-v1.5:** [BAAI, MIT license](#), run with [fastembed](#)
- **FiQA-2018, SciFact, NFCorpus, SCIDOCS** via [BEIR](#) (Thakur et al., NeurIPS 2021); Dave Ebbelaar, [hybrid-retrieval tutorial](#) (the FiQA + NDCG approach this lab moves into PostgreSQL)
- **This talk:** [hybrid-lab/sql/](#), [hybrid-lab/results/](#), [hybrid-search-plugin/](#)

Measured on PostgreSQL 18.6, pgvector 0.8.6, pg\_textsearch 1.4.0, September 2026